

Part 2 Dynamic Programming

Most of the content is from book

Algorithms Illuminated Part 3

Independent set

- **Definition:** $v_0 - v_1 - v_2 - v_3$

If v_1 is chosen, you can not choose v_0 or v_2 .

Solution set $S = \{v_1, v_3\}$ or $\{v_0, v_2\}$ or $\{v_0, v_3\}$

Independent set

- Input: a line graph like this: $v_0 - v_1 - v_2 - v_3$
- Output: a subset of the input that can be used to reach all vertices

Independence **Constraint**:

If v_1 is chosen, you can not choose its neighbors v_0 or v_2 .

Solution set $S = \{v_1, v_3\}$ or $\{v_0, v_2\}$ or $\{v_0, v_3\}$

Independent set

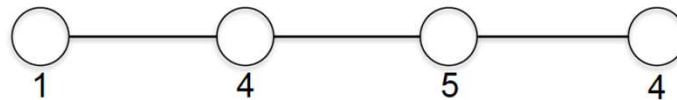
- Input: a line graph like this: $v_0 - v_1 - v_2 - v_3$
- Output: a subset of the input that can be used to reach all vertices

Can subset with v_1 and v_2 vertices be an answer?

Can v_2 be an answer?

Maximum Independent set

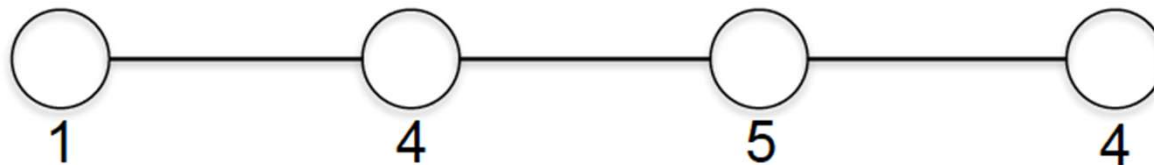
- Max independent set
add weights of selected vertices



Problem: Weighted Independent Set (WIS)

Input: An undirected graph $G = (V, E)$ and a nonnegative weight w_v for each vertex $v \in V$.

Output: An independent set $S \subseteq V$ of G with the maximum-possible sum $\sum_{v \in S} w_v$ of vertex weights.



Weights

1, 5

4, 4

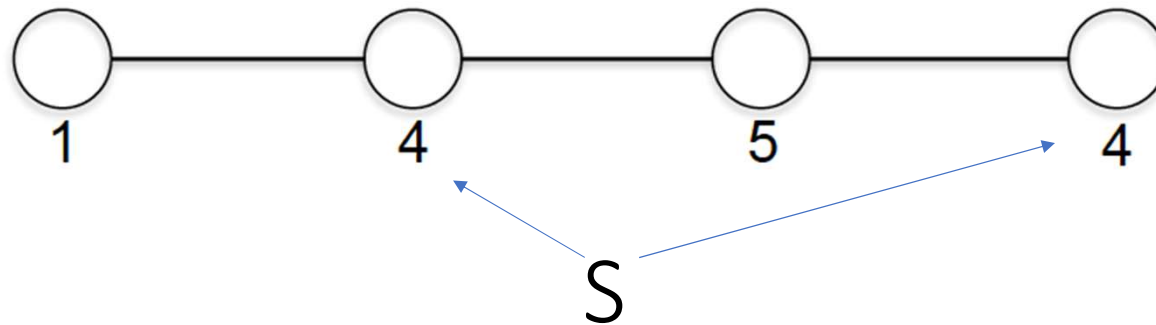
1, 4

4, 5 not allowed

Problem: Weighted Independent Set (WIS)

Input: An undirected graph $G = (V, E)$ and a nonnegative weight w_v for each vertex $v \in V$.

Output: An independent set $S \subseteq V$ of G with the maximum-possible sum $\sum_{v \in S} w_v$ of vertex weights.



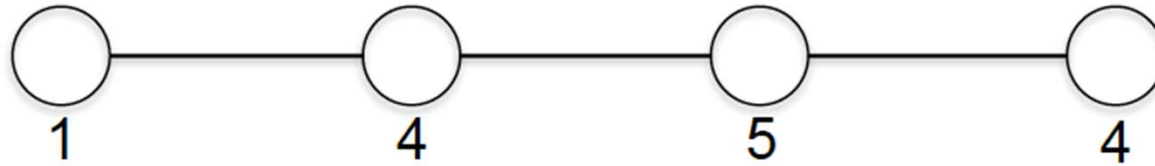
Weights

1, 5

4, 4

1, 4

W = 8



- Vertices are courses
- Weight on vertex represent credit units
- Edges represent conflicts between courses
- Max independent set gives you feasible course schedule with heaviest load (in credit units)

1, 5

4, 4

1, 4

Example

If vertices represent people and edges pairs of people who dislike each other, the independent sets correspond to groups of people who all get along.

Pop Quiz

- Solve Maximum Weight Independent set problem



Strategies

- Greedy
- Divide and Conquer
- Dynamic Programming

Greedy Weighted Independent Set



WIS: A Greedy Approach

```
 $S := \emptyset$   
sort vertices of  $V$  by weight  
for each  $v \in V$ , in nonincreasing order of weight do  
    if  $S \cup \{v\}$  is an independent set of  $G$  then  
         $S := S \cup \{v\}$   
return  $S$ 
```

Quiz 16.2

What is the total weight of the output of the greedy algorithm when the input graph is the four-vertex path on page 105? Is this the maximum possible?

- a) 6; no
- b) 6; yes
- c) 8; no
- d) 8; yes

(See Section 16.1.4 for the solution and discussion.)

Warning about greedy design pattern

- Greedy algorithms are usually not correct.
- Sometimes not optimal
- Your mom is correct: Don't be always greedy

Divide and Conquer

WIS: A Divide-and-Conquer Approach

$G_1 :=$ first half of G

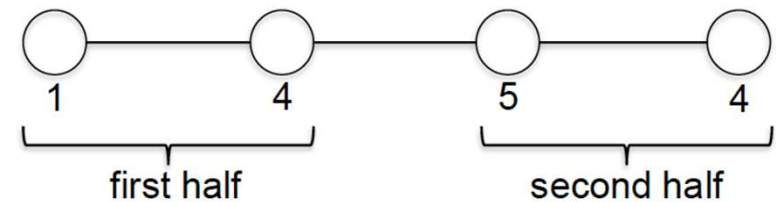
$G_2 :=$ second half of G

$S_1 :=$ recursively solve the WIS problem on G_1

$S_2 :=$ recursively solve the WIS problem on G_2

combine S_1, S_2 into a solution S for G

return S



Does divide and conquer work here?

WIS: A Divide-and-Conquer Approach

$G_1 :=$ first half of G

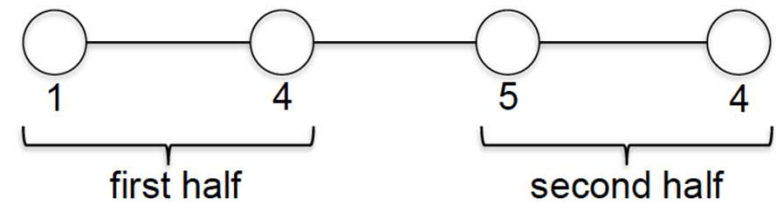
$G_2 :=$ second half of G

$S_1 :=$ recursively solve the WIS problem on G_1

$S_2 :=$ recursively solve the WIS problem on G_2

combine S_1, S_2 into a solution S for G

return S



Does divide and conquer work here?

WIS: A Divide-and-Conquer Approach

$G_1 :=$ first half of G

$G_2 :=$ second half of G

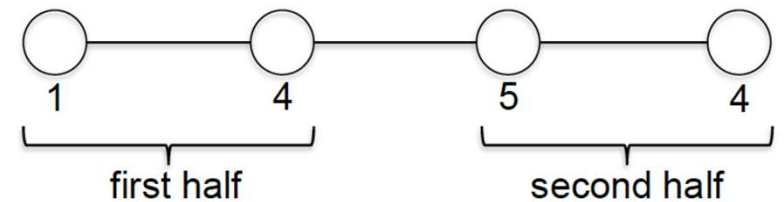
$S_1 :=$ recursively solve the WIS problem on G_1

$S_2 :=$ recursively solve the WIS problem on G_2

combine S_1, S_2 into a solution S for G

return S

- First half returns 4
- Second half returns 5
- Not feasible



Strategies

Greedy

Divide and Conquer

- Dynamic Programming

Strategies

~~Greedy~~

Don't be greedy

~~Divide and Conquer~~

Unite, don't divide

- Dynamic Programming

Dynamic Programming Mantra

An optimal solution must be constructed
from optimal solutions to smaller subproblems,
thereby narrowing down the field of candidates
to a manageable number

Find max independent set S with weight W



- Answer Set S ?
- Weight W of vertices in S ?

Find max independent set S with weight W



- Answer Set S ?
- Weight W of vertices in S ?

Max Wt Independent Set

More concretely, let $G = (V, E)$ denote the n -vertex path graph with edges $(v_1, v_2), (v_2, v_3), \dots, (v_{n-2}, v_{n-1}), (v_{n-1}, v_n)$ and a nonnegative weight w_i for each vertex $v_i \in V$. Assume that $n \geq 2$; otherwise, the answer is obvious. Suppose we magically knew an MWIS $S \subseteq V$ with total weight W . What can we say about it? Here's a tautology: S either contains the final vertex v_n , or it doesn't. Let's examine these cases in reverse order.

What if we knew the solution, set S and wt W

Two cases

1) Last vertex v_n not in solution



2) Last vertex v_n in solution



Case 1



- Last vertex v_n **not** in solution

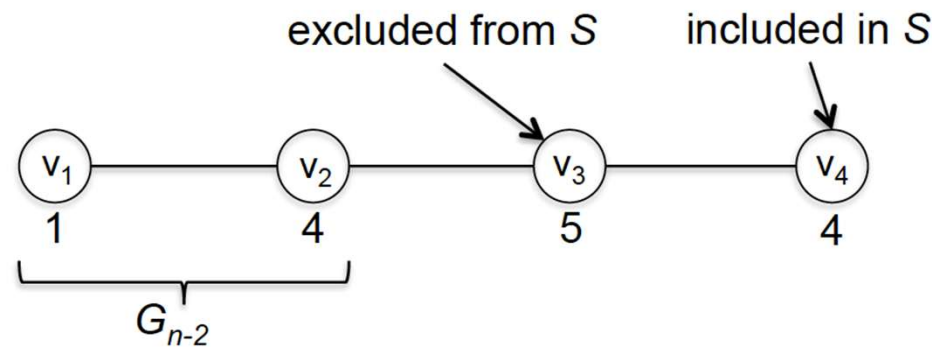


- For G_n , Max Independent set is S and weight is W
- Find S and W for G_{n-1} ?

For G_{n-1} , S is also a maximum wt. independent set with weight W

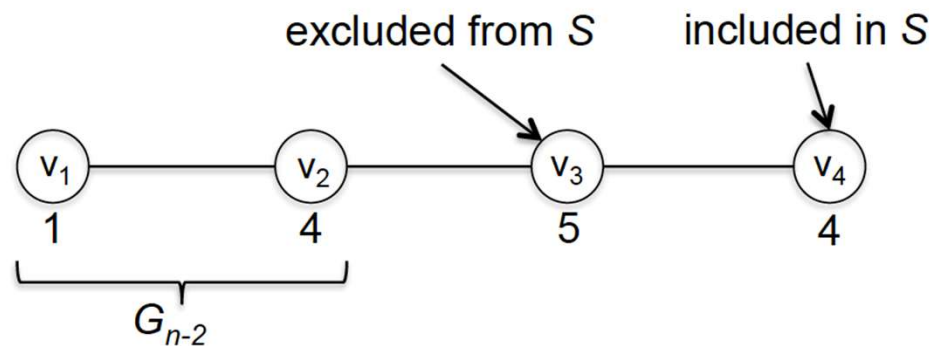
Case 2

Case 2: $v_n \in S$. Suppose S includes the last vertex v_n . As an independent set, S cannot include two consecutive vertices from the path, so it excludes the penultimate vertex: $v_{n-1} \notin S$. Obtain the $(n - 2)$ -vertex path graph G_{n-2} from G by plucking off the last two vertices and edges:³



Case 2

Case 2: $v_n \in S$. Suppose S includes the last vertex v_n . As an independent set, S cannot include two consecutive vertices from the path, so it excludes the penultimate vertex: $v_{n-1} \notin S$. Obtain the $(n-2)$ -vertex path graph G_{n-2} from G by plucking off the last two vertices and edges:³



- For G_{n-2} , $S - v_n$ is the independent set, with weight $W - w_n$

In other words, once you know that an MWIS includes the last vertex, you know exactly what it looks like: It's an MWIS of the smaller graph G_{n-2} , supplemented with the final vertex v_n . Summarizing, two and only two candidates are vying to be an MWIS:

Lemma 16.1 (WIS Optimal Substructure) *Let S be an MWIS of a path graph G with at least 2 vertices. Let G_i denote the subgraph of G comprising its first i vertices and $i - 1$ edges. Then, S is either:*

(i) an MWIS of G_{n-1} ; or

(ii) an MWIS of G_{n-2} , supplemented with G 's final vertex v_n .

Lemma 16.1 singles out the only two possibilities for an MWIS, so whichever option has larger total weight is an optimal solution. We therefore have a recursive formula—a *recurrence*—for the total weight of an MWIS:

Corollary 16.2 (WIS Recurrence) *With the assumptions and notation of Lemma 16.1, let W_i denote the total weight of an MWIS of G_i . (When $i = 0$, interpret W_i as 0.) Then*

$$W_n = \max\{\underbrace{W_{n-1}}_{\text{Case 1}}, \underbrace{W_{n-2} + w_n}_{\text{Case 2}}\}.$$

More generally, for every $i = 2, 3, \dots, n$,

$$W_i = \max\{W_{i-1}, W_{i-2} + w_i\}.$$

A Recursive Algorithm for WIS

Input: a path graph G with vertex set $\{v_1, v_2, \dots, v_n\}$
and a nonnegative weight w_i for each vertex v_i .

Output: a maximum-weight independent set of G .

```
1 if  $n = 0$  then                                // base case #1
2     return the empty set
3 if  $n = 1$  then                                  // base case #2
4     return  $\{v_1\}$ 
   // recursion when  $n \geq 2$ 
5  $S_1 :=$  recursively compute an MWIS of  $G_{n-1}$ 
6  $S_2 :=$  recursively compute an MWIS of  $G_{n-2}$ 
7 return  $S_1$  or  $S_2 \cup \{v_n\}$ , whichever has higher weight
```

Verify

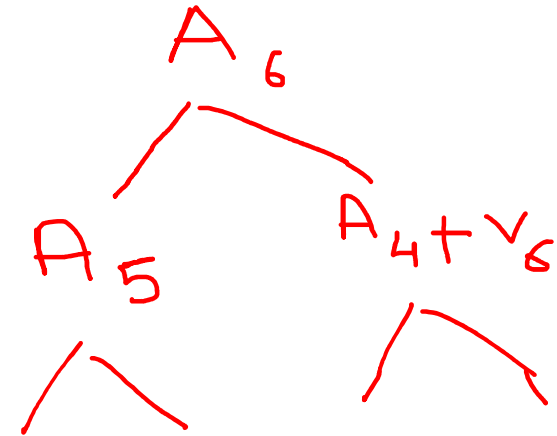
For example, for the input graph



you should check that the final array values are:

prefix length i						
0	1	2	3	4	5	6
0	3	3	4	9	9	14

0 0 0 0 0 0
 3 2 1 6 4 5 Wk.
 v_1 v_2 v_3 v_4 v_5 v_6

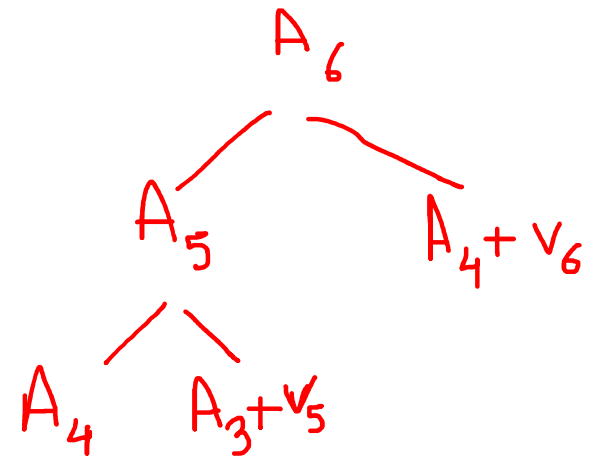


3	2	1	6	4	5
v_1	v_2	v_3	v_4	v_5	v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = ?$$

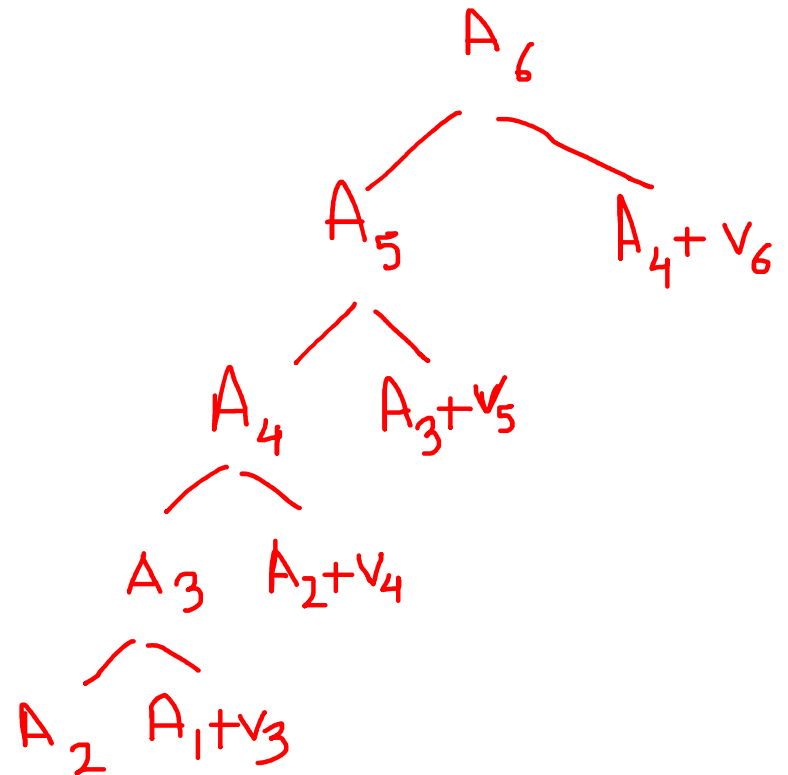


3	2	1	6	4	5
v_1	v_2	v_3	v_4	v_5	v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = ?$$

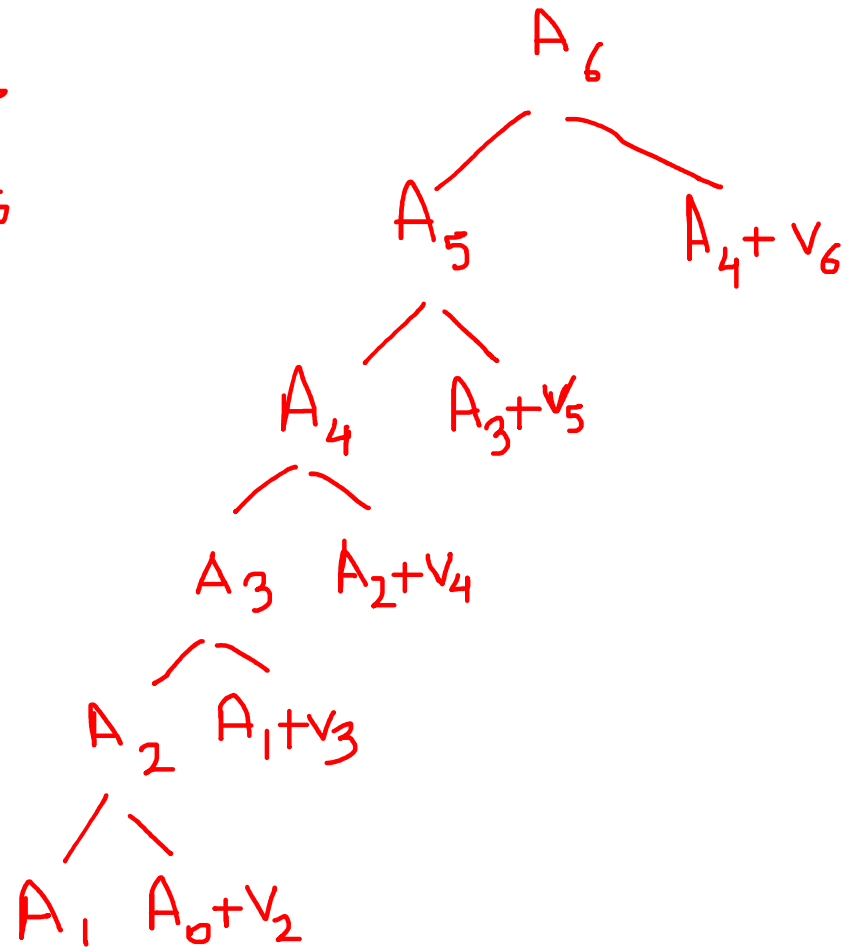


3	2	1	6	4	5
v_1	v_2	v_3	v_4	v_5	v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = ?$$

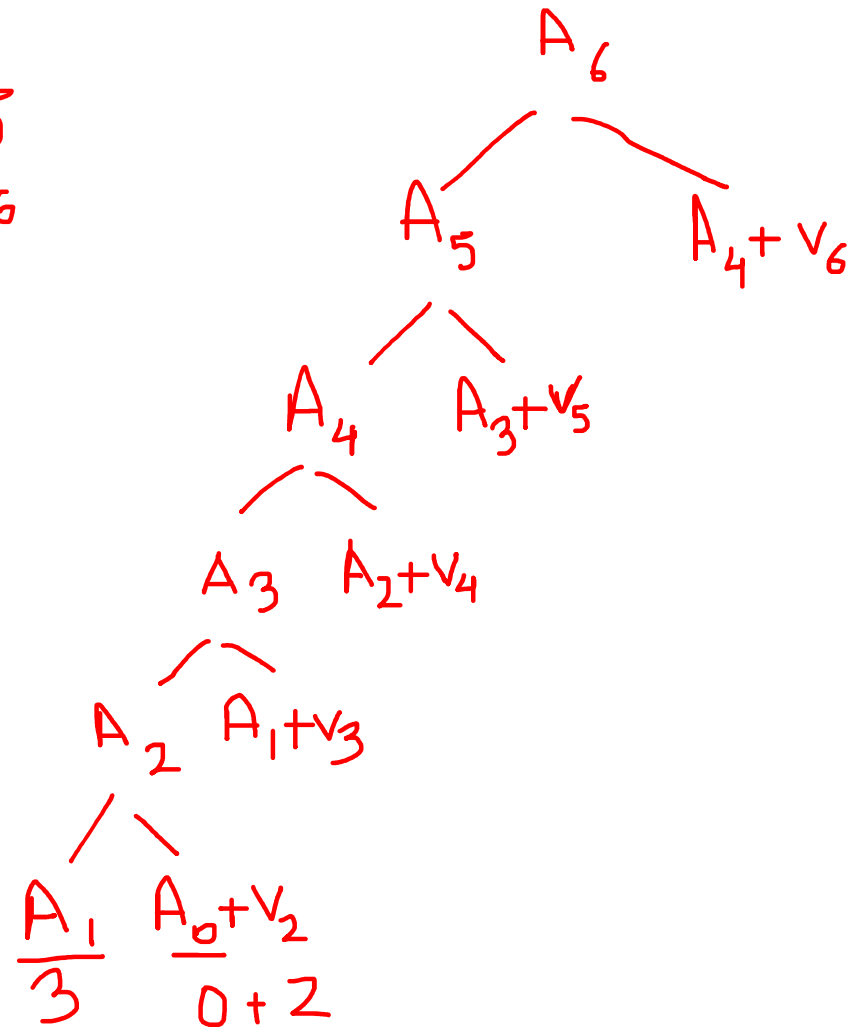


3	2	1	6	4	5
v_1	v_2	v_3	v_4	v_5	v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = ?$$

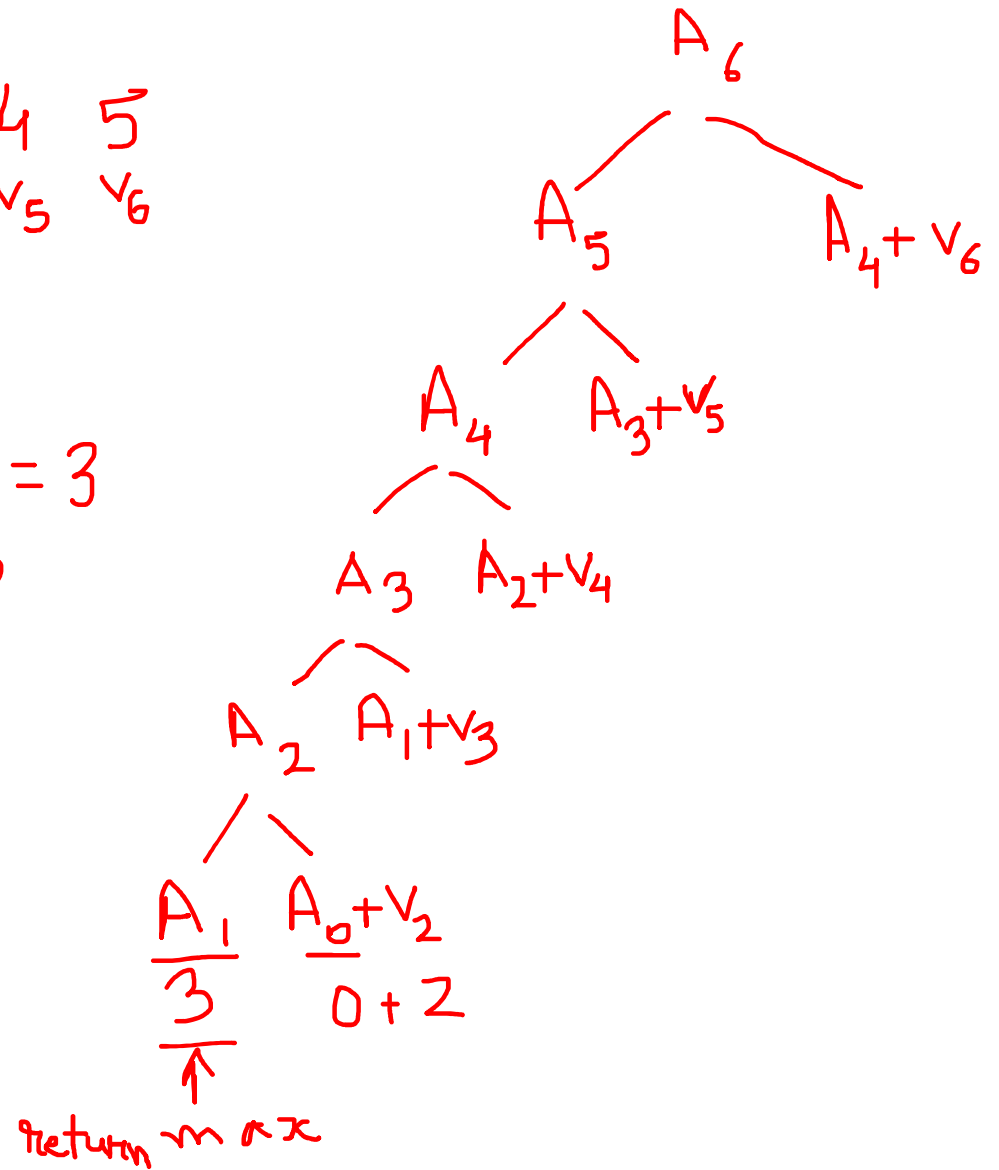


3 2 1 6 4 5
 v_1 v_2 v_3 v_4 v_5 v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = ?$$



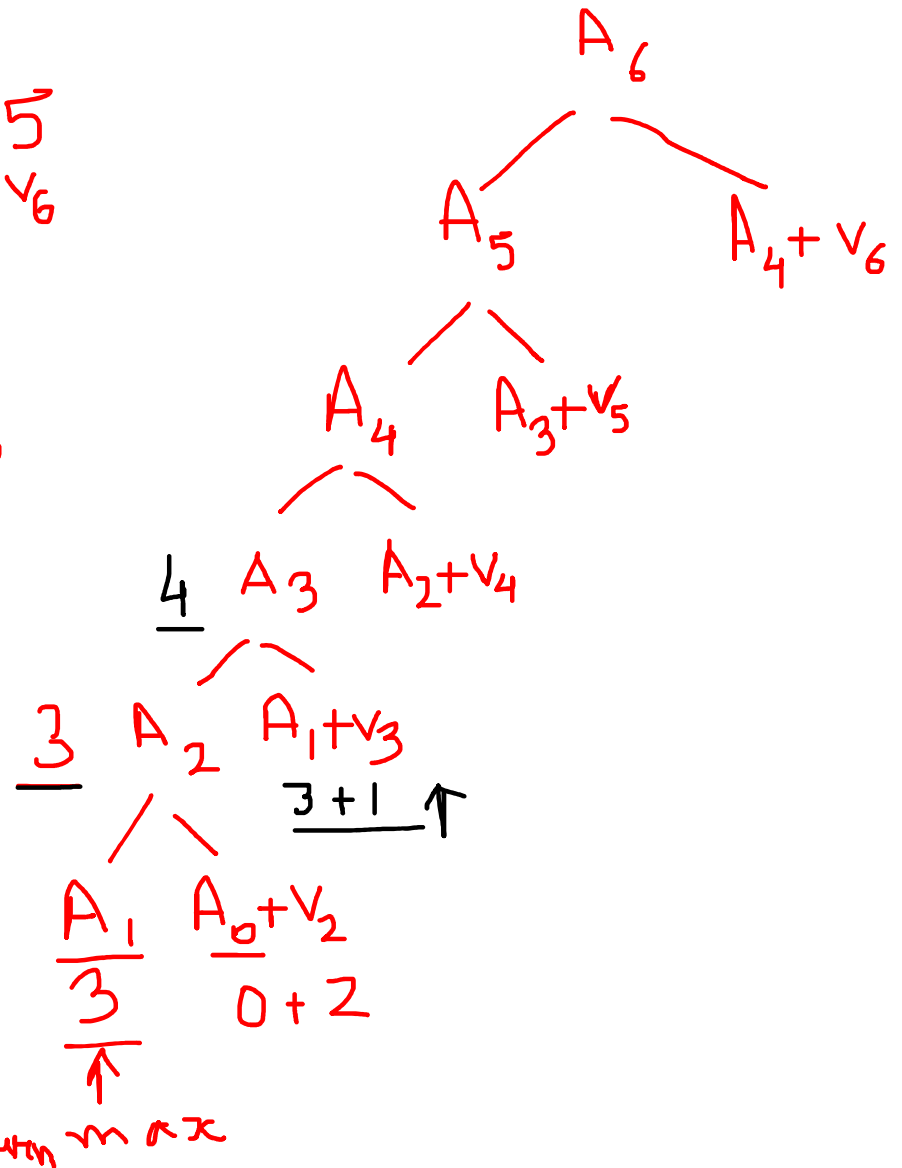
3 2 1 6 4 5
 v_1 v_2 v_3 v_4 v_5 v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = 3$$

$$A_3 = 4$$



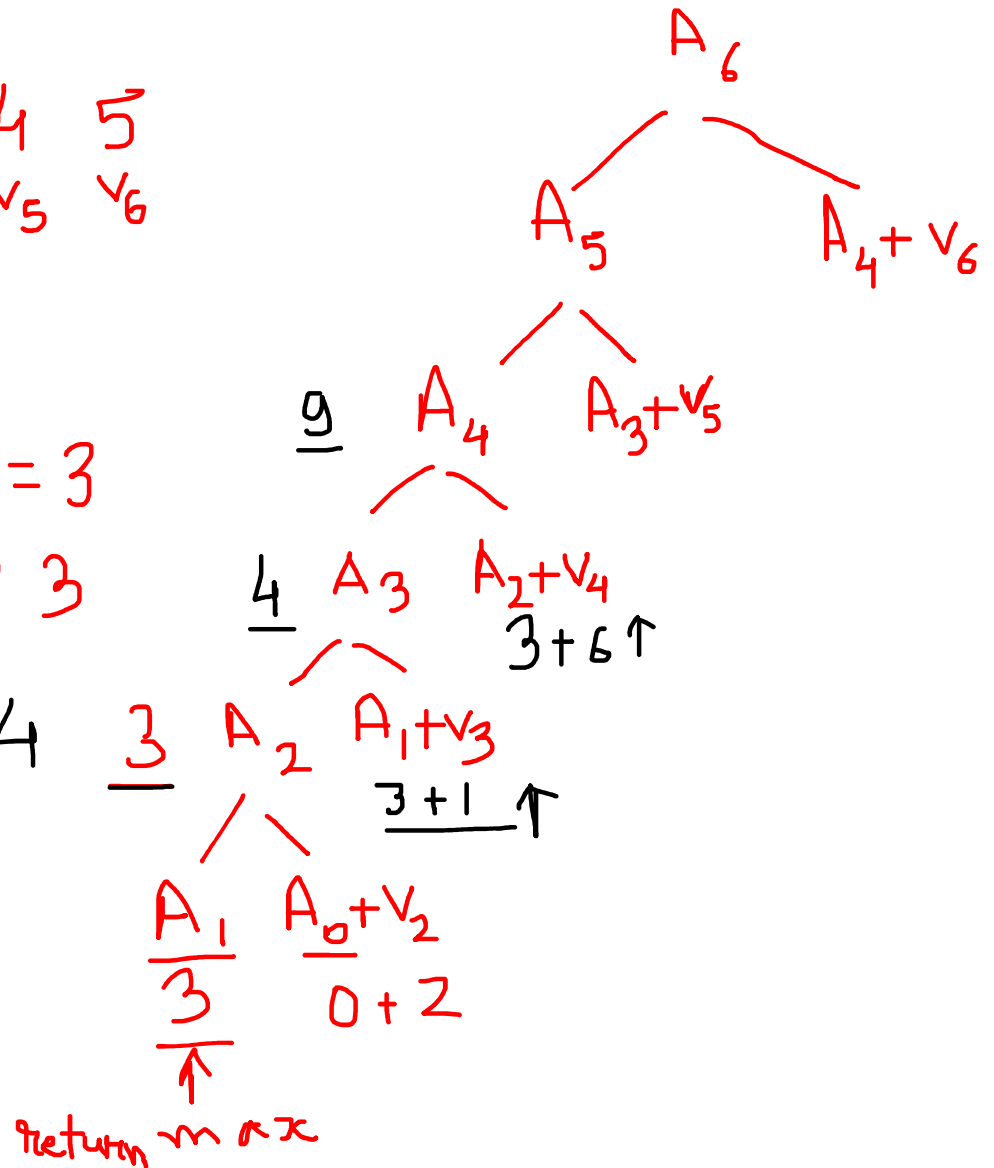
3 2 1 6 4 5
 v_1 v_2 v_3 v_4 v_5 v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = 3$$

$$A_3 = 4$$



3 2 1 6 4 5
 v_1 v_2 v_3 v_4 v_5 v_6

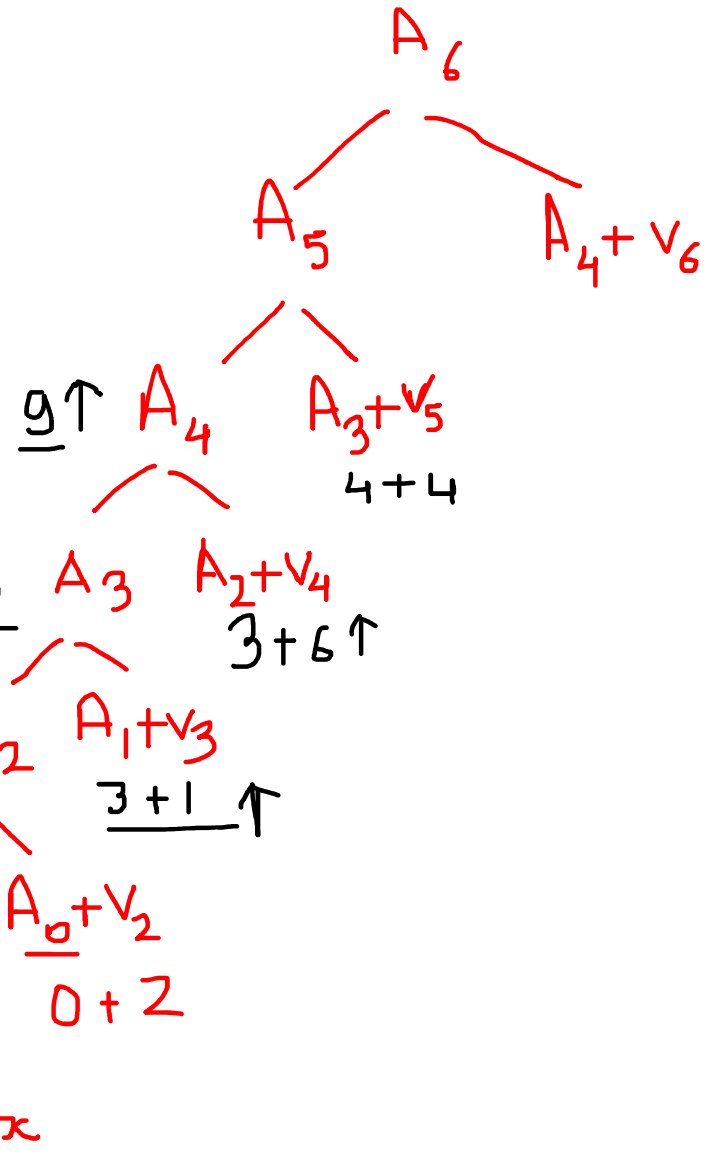
$$A_0 = 0$$

$$A_1 = v_1 = 3$$

$$A_2 = 3$$

$$A_3 = 4$$

$$A_4 = 9$$



3 2 1 6 4 5
 v_1 v_2 v_3 v_4 v_5 v_6

$$A_0 = 0$$

$$A_1 = v_1 = 3$$

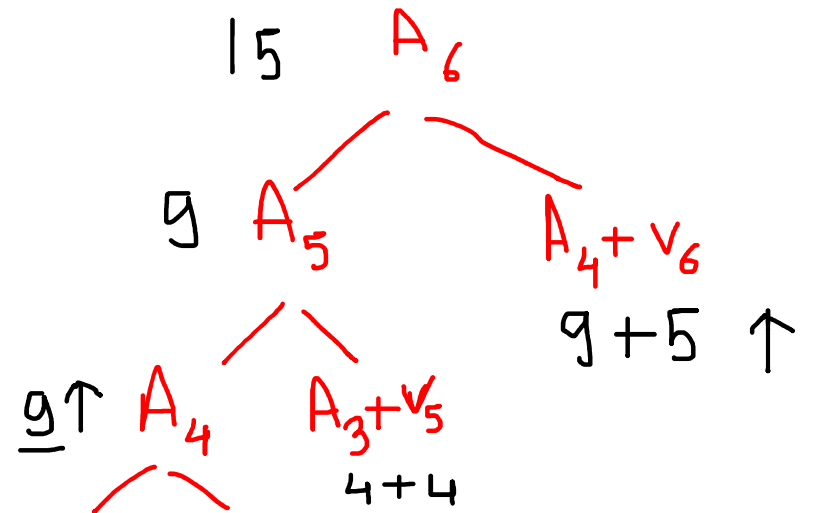
$$A_2 = 3$$

$$A_3 = 4$$

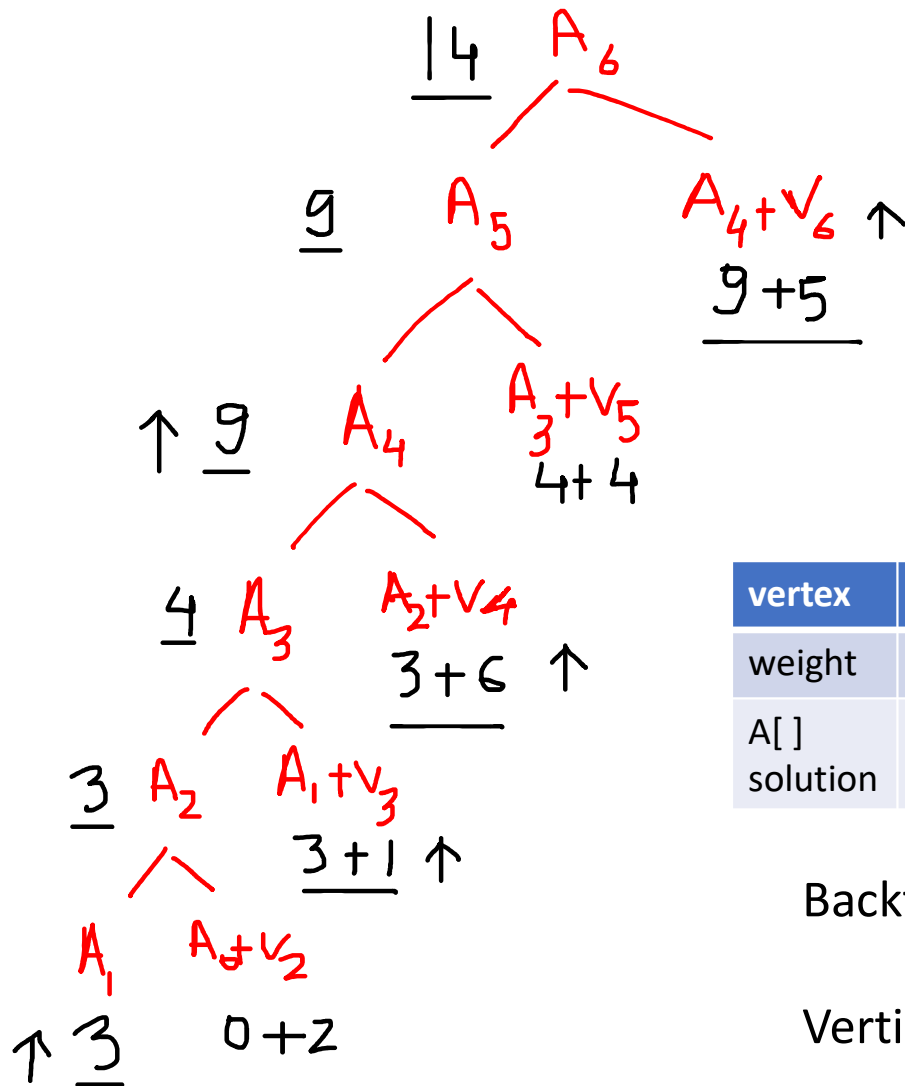
$$A_4 = 9$$

$$A_5 = 9$$

$$A_6 = 14$$



return max



Reconstruct
answer

vertex	v1	v2	v3	v4	v5	v6
weight	3	2	1	6	4	5
A[] solution	3	3	4	9	9	14

Backtrack from root A_6 to leaf

Vertices in solution set: v1, v4, v6

Quiz 16.3

What is the asymptotic running time of the recursive WIS algorithm, as a function of the number n of vertices? (Choose the strongest correct statement.)

- a) $O(n)$
- b) $O(n \log n)$
- c) $O(n^2)$
- d) none of the above

Solution to Quiz 16.3

Correct answer: (d). Superficially, the recursion pattern looks similar to that of $O(n \log n)$ -time divide-and-conquer algorithms like MergeSort, with two recursive calls followed by an easy combine step. But there's a big difference: The MergeSort algorithm throws away half the input before recursing, while our recursive WIS algorithm throws away only one or two vertices (perhaps out of thousands or millions). Both algorithms have recursion trees with branching factor 2.⁷ The former has roughly $\log_2 n$ levels and, hence, only a linear number of leaves. The latter has no leaves until levels $n/2$ and later, which implies that it has at least $2^{n/2}$ leaves. We conclude that the running time of the recursive algorithm grows exponentially with n .

Quiz 16.4

Each of the (exponentially many) recursive calls of the recursive WIS algorithm is responsible for computing an MWIS of a specified input graph. Ranging over all of the calls, how many *distinct* input graphs are ever considered?

a) $\Theta(1)^5$

b) $\Theta(n)$

c) $\Theta(n^2)$

d) $2^{\Theta(n)}$

Solution to Quiz 16.4

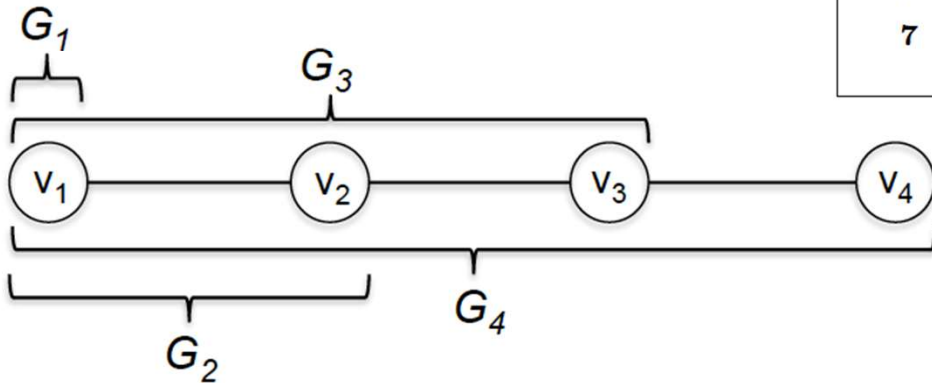
Correct answer: (b).

A Recursive Algorithm for WIS

Input: a path graph G with vertex set $\{v_1, v_2, \dots, v_n\}$
and a nonnegative weight w_i for each vertex v_i .

Output: a maximum-weight independent set of G .

```
1 if  $n = 0$  then                                // base case #1
2   return the empty set
3 if  $n = 1$  then                                  // base case #2
4   return  $\{v_1\}$ 
  // recursion when  $n \geq 2$ 
5  $S_1 :=$  recursively compute an MWIS of  $G_{n-1}$ 
6  $S_2 :=$  recursively compute an MWIS of  $G_{n-2}$ 
7 return  $S_1$  or  $S_2 \cup \{v_n\}$ , whichever has higher weight
```



Memoize the recursive algorithm

- From exponential-time to linear-time
- Top Down
- Bottom Up

Verify

For example, for the input graph



you should check that the final array values are:

prefix length i						
0	1	2	3	4	5	6
0	3	3	4	9	9	14

Principles of Dynamic Programming

- Three step recipe

The Dynamic Programming Paradigm

1. Identify a relatively small collection of subproblems.
2. Show how to quickly and correctly solve “larger” subproblems given the solutions to “smaller” ones.
3. Show how to quickly and correctly infer the final solution from the solutions to all of the subproblems.

Analyzing running time of Dynamic Programming

For example, suppose an algorithm solves at most $f(n)$ different subproblems (working systematically from “smallest” to “largest”), using at most $g(n)$ time for each, and performs at most $h(n)$ postprocessing work to extract the final solution (where n denotes the input size). The algorithm’s running time is then at most

$$\underbrace{f(n)}_{\text{\# subproblems}} \times \underbrace{g(n)}_{\substack{\text{time per subproblem} \\ \text{(given previous solutions)}}} + \underbrace{h(n)}_{\text{postprocessing}}. \quad (16.1)$$

The three steps of the recipe call for keeping $f(n)$, $g(n)$, and $h(n)$, respectively, as small as possible. In the basic **WIS** algorithm, without the **WIS Reconstruction** postprocessing step, we have $f(n) = O(n)$, $g(n) = O(1)$, and $h(n) = O(1)$, for an overall running time of $O(n)$. If we include the reconstruction step, the $h(n)$ term jumps to $O(n)$, but the overall running time $O(n) \times O(1) + O(n) = O(n)$ remains linear.

Dynamic Programming vs Divide and Conquer

1. Each recursive call of a typical divide-and-conquer algorithm commits to a single way of dividing the input into smaller subproblems.¹⁰ Each recursive call of a dynamic programming algorithm keeps its options open, considering multiple ways of defining smaller subproblems and choosing the best of them.¹¹
2. Because each recursive call of a dynamic programming algorithm tries out multiple choices of smaller subproblems, subproblems generally recur across different recursive calls; caching subproblem solutions is then a no-brainer optimization. In most divide-and-conquer algorithms, all the subproblems are distinct and there's no point in caching their solutions.¹²

4. In a divide-and-conquer algorithm, subproblems are chosen primarily to optimize the running time; correctness often takes care of itself.¹⁴ In dynamic programming, subproblems are usually chosen with correctness in mind, come what may with the running time.¹⁵
5. Relatedly, a divide-and-conquer algorithm generally recurses on subproblems with size at most a constant fraction (like 50%) of the input. Dynamic programming has no qualms about recursing on subproblems that are barely smaller than the input (like in the WIS algorithm), if necessary for correctness.

Differences continued

6. The divide-and-conquer paradigm can be viewed as a special case of dynamic programming, in which each recursive call chooses a fixed collection of subproblems to solve recursively. As the more sophisticated paradigm, dynamic programming applies to a wider range of problems than divide-and-conquer, but it is also more technically demanding to apply (at least until you've had sufficient practice).

Dynamic Programming vs Divide and Conquer

- First try divide and conquer

If merging subproblems looks tedious,
subproblems do not help in quick merging,
try Dynamic Programming

Story behind the name

- Dynamic Programming
- Rand Corporation, Researcher: Bellman
- Angry Senator, anti-research

The first point of confusion is the anachronistic use of the word “programming.” In modern times it refers to coding, but back in the 1950s “programming” usually meant “planning.” (For example, it has this meaning in the phrase “television programming.”) What about “dynamic?” For the full story, I refer you to the father of dynamic programming himself, Richard E. Bellman, writing about his time working at the RAND Corporation:

The 1950’s were not good years for mathematical research. We had a very interesting gentleman in Washington named Wilson. He was Secretary of Defense, and he actually had a pathological fear and hatred of the word, research. I’m not using the term lightly; I’m using it precisely. His face with suffuse, he would turn red, and he would get violent if people used the term, research, in his presence. You can imagine how he felt, then, about the term, mathematical. The RAND Corporation was employed by the Air Force, and the Air Force had Wilson as its boss, essentially.

Hence, I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics inside the RAND Corporation. What title, what name, could I choose? In the first place I was interested in planning, in decision making, in thinking. But planning, is not a good word for various reasons. I decided therefore to use the word, “programming.” . . . [“Dynamic”] has a very interesting property as an adjective, and that is it’s impossible to use the word, dynamic, in the pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It’s impossible. Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to. So I used it as an umbrella for my activities.¹⁶